UNIVERSITI TUNKU ABDUL RAHMAN

ACADEMIC YEAR 2018/2019

SEPTEMBER EXAMINATION

UECS3253/UECS3463 WIRELESS APPLICATION DEVELOPMENT

FRIDAY, 21 SEPTEMBER 2018 TIME : 2.30 PM – 4.30 PM (2 HOURS)

BACHELOR OF SCIENCE (HONS) SOFTWARE ENGINEERING
BACHELOR OF ENGINEERING (HONS) ELECTRONICS (COMPUTER NETWORKING)

Instruction to Candidates :

This question paper consists of THREE (3) questions in Section A and TWO (2) questions in Section B.

Section A: Answer ALL questions.

Section B: Answer ONE (1) question only.

Each question carries 25 marks.

Note: Should a candidate answer more than ONE (1) question in Section B, marks will only be awarded for the FIRST question in Section B in the order the candidate submits the answer.

Answer the questions in the answer booklet provided.

UECS3253/UECS3463 WIRELESS APPLICATION DEVELOPMENT

SECTION A: Answer **ALL THREE (3)** questions in this section.

Q1. (a) Mobile apps can be classified broadly into the categories of *native apps*, *mobile web apps* and *hybrid apps*.

List **THREE (3)** key features of *native apps* and identify a particular platform / programming language for a native app. (8 marks)

(b) Figure 1.1 shows the output from a simple mobile app while Figure 1.2 shows an outline of the class used to generate the app. The following are the relevant features of its implementation:

- The three sentences immediately below the title with the string ***Subject Lists*** are generated from a user defined component named **Subject**.
- The name of the subject (***Science***) is contained in the **Subject** component state.
- The respective marks, i.e. ***45***, ***90*** and ***60***, are passed as props to the **Subject** component.
- The styling of the appearance of the title and the three lines of text immediately below is done using the **StyleSheet** API.

(i) Complete the implementation for the **Subject** component. (7 marks)

(ii) Complete the implementation for the **SubjectList** component. (10 marks)

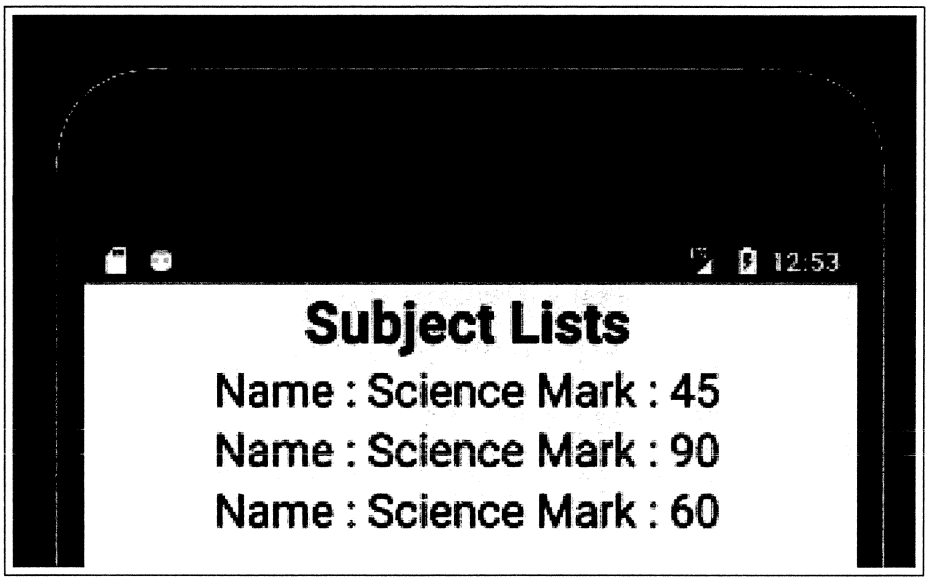

Figure 1.1

Q1(b). (Continued)

```
import React, { Component } from 'react';
import {
        Text,
        View,
        StyleSheet
} from 'react-native';

class Subject extends Component {

        // Implement code for Q1 (b) (i)


}

export default class SubjectList extends Component {

        // Implement code for Q1 (b) (ii)
}
```

Figure 1.2

[Total : 25 marks]

Q2. Table 2.1 shows the sample data of a table named **animals** in an SQLite database named **animalsDb** that is used in a mobile app of an Australian wildlife sanctuary to provide information of their resident animals to users.

id	name	animalType	gender	birthDate
1	Clara	Koala	Female	1485595500
2	Hannah	Koala	Female	1360491010
3	Boxer	Kangaroo	Male	1217568900
4	Mimi	Quokka	Female	1518757754

Table 2.1: Sample data of *animals* table

(a) New data of animals are downloaded by the mobile app to be inserted into the abovementioned **animals** table. The data is in JSON format and is available in a variable named **newAnimals**. Figure 2.2 shows the sample data in JSON format.

```
[
    {
        "animalType": "Koala",
        "birthDate": 1485595500,
        "gender": "Female",
        "name": "Clara"
    },
    {
        "animalType": "Koala",
        "birthDate": 1360491010,
        "gender": "Female",
        "name": "Hannah"
    },
    {
        "animalType": "Kangaroo",
        "birthDate": 1217568900,
        "gender": "Male",
        "name": "Boxer"
    },
    {
        "animalType": "Quokka",
        "birthDate": 1518757754,
        "gender": "Female",
        "name": "Mimi"
    }
]
```

Figure 2.2: Sample JSON data

Construct code segment to insert new rows to the **animals** table using the JSON data in the **newAnimals** variable. (9 marks)

Q2. (Continued)

(b) In some situations, existing data of animals are updated due to errors. Updates to the data are downloaded by the app using the JSON format as shown in Figure 2.3 and is available in a variable named **updateAnimals**.

```
[
    {
        "id": 3,
        "animalType": "Kangaroo",
        "birthDate": 1217568900,
        "gender": "Male",
        "name": "Boxassura"
    },
    {
        "id": 4,
        "animalType": "Quokka",
        "birthDate": 1518757754,
        "gender": "Female",
        "name": "Mi Mi"
    }
]
```

Figure 2.3: Sample JSON data

Construct code segment to update the existing rows in **animals** using the JSON data in the **updateAnimals** variable. (7 marks)

(c) A screen of the mobile app displays a **FlatList** listing the names of all animals based on the specified **animalType**.

Constuct code segment to query the **animals** table by **animalType** column and prepare the data as a JavaScript object that is suitable to populate the **FlatList** with the names of the animals. Assume that the selected **animalType** is available in a variable named **selectedAnimalType**. (9 marks)

[Total : 25 marks]

Q3. (a) **AsyncStorage** is a basic option for persisting data from a React Native app. Provide **ONE (1)** advantage and **TWO (2)** disadvantages of this approach for data persistence. (6 marks)

(b) Figures 3.1 to 3.5 show the various outputs from a mobile app while Figure 3.6 shows the outline of the component class used to implement it. The functionalities implemented are as follows:

- Clicking on a list item that is not yet stored in **AsyncStorage** will store it in **AsyncStorage**.
- Clicking on a list item that already exists in **AsyncStorage** will remove it from **AsyncStorage**.
- The current action will indicate either an addition or removal of an item.
- The number of items saved will dynamically change to reflect addition / removal of items to / from **AsyncStorage**.
- When the app is reloaded, the number of items saved in **AsyncStorage** will be displayed along with their names.

Figure 3.1 shows the initial startup screen for the app. After the user presses the **a : dog**, **c : mouse** and **e : horse** items consecutively, the display is as shown in Figure 3.2.

When the user presses on **c : mouse** again, the display is as shown in Figure 3.3. Then the user presses on **b : cat** which results in Figure 3.4. Finally the user reloads the app resulting in the display shown in Figure 3.5.

Provide the necessary implementation in Figure 3.6 in order to accomplish the functionality described. The implementation should include the appropriate calls to the **AsyncStorage** component. (19 marks)

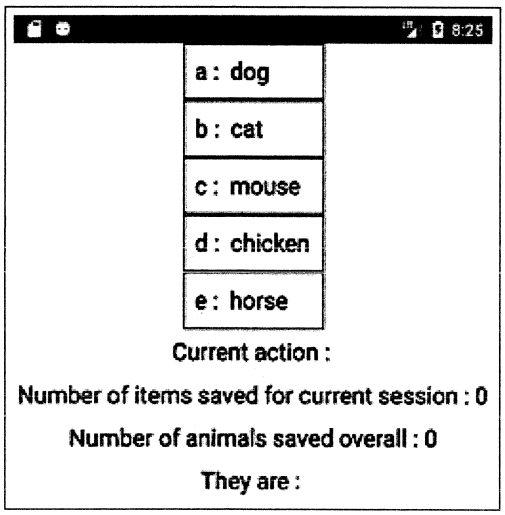

Figure 3.1

Q3(b). (Continued)

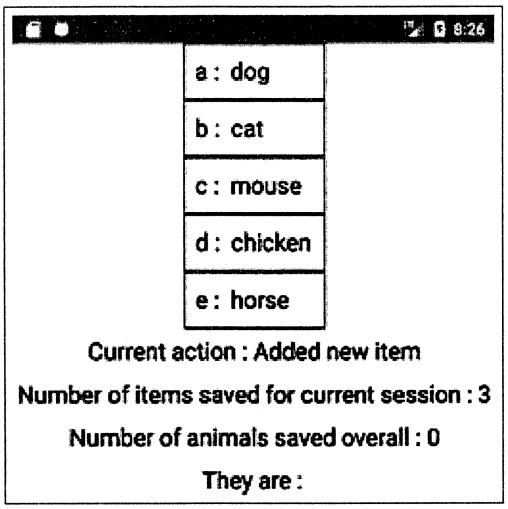

Figure 3.2

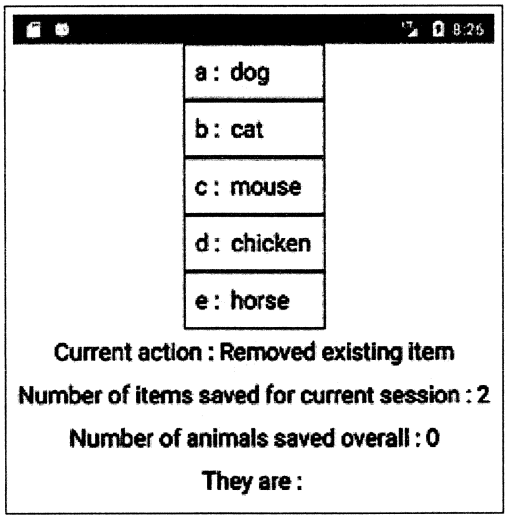

Figure 3.3

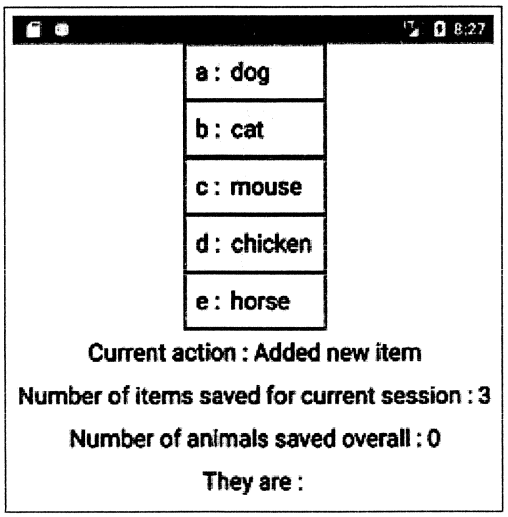

Figure 3.4

Q3(b). (Continued)

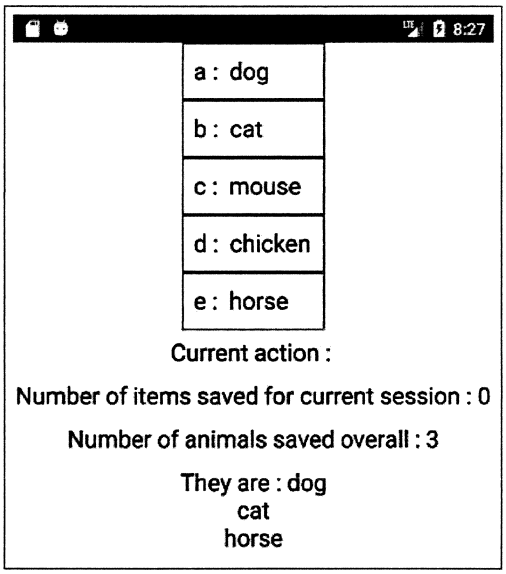

Figure 3.5

```
export default class SimpleList extends Component {

  constructor(props) {
    super(props);
    this.state = {
      numItems : 0,
      numKeys : 0,
      storedAnimalList : '',
      currentAction : '',
      data: [
        { key: "a", animal: "dog" },
        { key: "b", animal: "cat" },
        { key: "c", animal: "mouse" },
        { key: "d", animal: "chicken" },
        { key: "e", animal: "horse" }
      ]
    };
  }

  async componentDidMount() {
        // Implement code for Q3 (b)
  }

  async pressInHandler(data) {
        // Implement code for Q3 (b)
  }

  pressOutHandler = () => {
      // No action necessary
  };

  // code continued on next page ...
```

Figure 3.6

Q3(b). (Continued)

```
  _renderItem = data => {
    return (
      <TouchableHighlight
        onPressIn={() => this.pressInHandler(data)}
        onPressOut={() => this.pressOutHandler}
      >
        <Text>{data.item.key} :  {data.item.animal}</Text>
      </TouchableHighlight>
    );
  };

  render() {
    return (
      <View>
          <FlatList
            data={this.state.data}
            renderItem={this._renderItem} />
          <Text>
            Current action : {this.state.currentAction}
          </Text>
          <Text>
            Number of items saved for current session :
            {this.state.numItems}
          </Text>
          <Text>
             Number of animals saved overall :
             {this.state.numKeys}
          </Text>
          <Text>
             They are : {this.state.storedAnimalList}
          </Text>
      </View>
    );
  }
}
```

Figure 3.6 (continued)

[Total : 25 marks]

SECTION B: Answer **ONLY ONE (1)** question from this section.

Q4. ZhuaChe is an e-hailing platform that allows a user to hail a car to travel from one place to another. The platform allows users to hail a car with the use of the customer app while drivers use the driver app to receive and accept requests.

(a) A person intended to join ZhuaChe as a driver is required to download the app for drivers register as a driver. The driver information is entered via the app and is sent via HTTP to a web server for processing. Upon processing, the web server responds to the app whether the registration is successful or otherwise.

Figure 4.1 shows a sample JSON data that is sent by the app to the web server during registration while Figure 4.2 shows a sample response from the web server.

```
{
    "name": "Ng Pei Li",
    "nric": "870620057792",
    "gender": "Female",
    "email": "ngpeili@hmail.com",
    "phone": "012-3456789",
    "address": "66 Jalan BU2/18, Bandar Utama",
    "city": "Petaling Jaya",
    "state": "Selangor",
    "drivingExperience": 12
}
```

Figure 4.1: Sample JSON data for driver registration

```
{
    "driverId": 2977,
    "email": "ngpeili@hmail.com",
    "success": true
}
```

Figure 4.2: Sample response

Constuct code segment to send the JSON data in Figure 4.1 to a web server at the URL **https://zhuache.my/api/driver/store** using HTTP POST method. Assume that the JSON data is already set to a variable named **driver**.

If the driver registration is successful, navigate the user to another screen using a route named **DriverHome**, passing the **driverId** and **email** as route parameters. If the driver registration is unsuccessful, display an alert with an appropriate error message.

Should any error occurs when attempting to connect and communicate with the web server, display an alert with an appropriate error message. (12 marks)

Q4. (Continued)

(b) A user who launches the customer app to request for a car will be shown a map indicating real time location of drivers in the vicinity. The app sends JSON data as shown in the sample in Figure 4.3 to the web server every 5 seconds. The server responds with JSON data containing the location of all drivers in the vicinity as shown in Figure 4.4.

```
{
    "customerId": 7543,
    "location": [
        3.139003,
        101.686854
    ]
}
```

Figure 4.3: Sample JSON data sent to server

```
[
    {
        "driverId": 2977,
        "location": [
            3.139103,
            101.683844
        ]
    },
    {
        "driverId": 8681,
        "location": [
            3.139998,
            101.683102
        ]
    }
]
```

Figure 4.4: Sample JSON data of drivers' location

Construct code segment to send the JSON data in Figure 4.3 every 5 seconds to the web server at the URL **https://zhuache.my/api/location/get** using HTTP POST method. Upon receiving the response as shown in Figure 4.4, the app updates the location of each driver in the vicinity on the map by calling the method **updateMap** for each driver, passing in two parameters - **driverId** and **location**. Assume that the data in Figure 4.3 has been set in a variable named **currentLocation**. (13 marks)

[Total : 25 marks]

Q5. MovieMania is a mobile app that provides information about movies. This app is required to make a *socket* connection using TCP to a server at **moviemania.my:8509** to send and receive data between the app and the server.

(a) Users of the app are required to sign in before accessing the information about movies. For the purpose of user sign in, the app sends user sign in information as shown in the sample JSON data in Figure 5.1.

The server responds to the app whether the sign in is successful or otherwise in JSON data as shown in the sample in Figure 5.2.

```
{
    "email": "chiakimhooi@hmail.com",
    "password": "*3Ya8!C_"
}
```

Figure 5.1: Sample user sign in data

```
{
    "id": 7453,
    "email": "chiakimhooi@hmail.com",
    "signedin": true
}
```

Figure 5.2: Sample server response

Construct code segment to open a TCP connection to the **WebSocket** URL at **moviemania.my:8509/signin** using the **WebSocket** API and send the user sign in data as shown in Figure 5.1 to the server. If the sign in is successful, navigate the user to another screen using a route named **Home**, supplying the **id** as route parameter.

If unsuccessful, set the state named **errorMessage** with an appropriate error message.

If a connection error occurs, display an alert with an appropriate error message.

Assume that the JSON data in Figure 5.1 is already set in a variable named **userSignIn**. (16 marks)

Q5. (Continued)

(b) Signed in users may search for movies by providing various search criteria. The server responds to the app by sending the results of the search in JSON format as shown in Figure 5.3 after receiving the search criteria from the app.

```
[
    {
        "id": 2351,
        "title": "The Contract",
        "lang": "Cantonese",
        "year": 1978,
        "casts": [
            "Michael Hui",
            "Sam Hui",
            "Ricky Hui"
        ]
    },
    {
        "id": 5995,
        "title": "Chicken & Duck Talk",
        "lang": "Cantonese",
        "year": 1988,
        "casts": [
            "Michael Hui",
            "Ricky Hui"
        ]
    }
]
```

Figure 5.3: Sample search result

Construct code segment to open a TCP connection to the **WebSocket** URL at **moviemania.my:8509/search** to send the search criteria which is in JSON format and has been set in a variable named **criteria**.

When the response is received, process the JSON data as shown in Figure 5.3 to populate a JavaScript object containing only the **title** and **year** of each movie.

(9 marks)

[Total : 25 marks]